ESGI Lyon

5ESGI AL, année 2026-2027

Clusterisation de conteneurs

Rapport des travaux dirigés

Ahmed Yahya LETAIEF

Novembre 2026

Sommaire

TD 1 : Dockerisation de My Favorite Places **3**

Exercice 1 : Dockerisation du projet . . . 3

1.1 Tester le serveur avec yarn dev et Bruno . . . 3

1.2 Dockeriser le serveur . . . 4

1.3 Créer un compose.yml pour le serveur et la BDD . . . 5

1.4 Dockeriser le client et l'ajouter au compose.yml . . . 6

1.4.a Dockerfile et lancement du client . . . 6

1.4.b Ajout au compose.yml et lancement . . . 7

1.5 Communication entre la BDD, le serveur et le front . . . 8

Exercice 2 : Démarrage, redémarrage et images légères . . . 8

2.1 Démarrer le serveur une fois la BDD prête . . . 8

2.2 Redémarrer automatiquement les services . . . 9

2.3 Optimiser les images avec les build stages . . . 9

2.3.a Serveur . . . 9

2.3.b Client . . . 9

2.3.c Résultat . . . 9

TD 2 : Cluster Docker Swarm **10**

Exercice 1 : DinD et DooD . . . 10

Prérequis : images sur un registry . . . 10

Exercice 2 : Création du cluster Docker Swarm . . . 10

2.1 Créer et lancer le manager et les 3 nœuds DinD . . . 10

2.2 Initialiser le cluster depuis le manager . . . 11

2.3 Faire rejoindre les nœuds comme workers . . . 12

2.4 Lister les nœuds depuis le manager . . . 12

Exercice 3 : Tests du cluster . . . 12

3.1 Créer la stack hello-world . . . 12

3.2 Copier la stack dans le manager . . . 13

3.3 Déployer la stack et observer les nœuds . . . 13

3.4 Faire varier la clause deploy . . . 14

3.4.a Sur le manager seulement (par rôle) . . . 14

3.4.b Sur tous les nœuds (mode global) . . . 15

TD 3 : Traefik et Portainer **16**

Exercice 1 : Déployer Traefik . . . 16

1.1 Relancer le cluster . . . 16

1.2 Déployer la stack Traefik sur le manager . . . 16

1.3 Accéder à Traefik depuis le navigateur . . . 16

Exercice 2 : Déployer une autre application . . . 18
2.1 Tester l'application avec Docker Compose . . . 18
2.2 Déployer l'application sur le cluster Swarm . . . 18
2.3 Publier le vote et les résultats avec Traefik . . . 19
Exercice 3 : Déployer Portainer . . . 20
3.1 Déployer Portainer sur le cluster . . . 20
3.2 Publier Portainer avec Traefik . . . 21
3.3 Ouvrir Portainer dans le navigateur . . . 21
Exercice 4 : Redéployer l'app de votes . . . 22
4.1 Supprimer la stack depuis le manager . . . 22
4.2 Republier la stack depuis Portainer . . . 22
4.3 Vérifier l'accès depuis le navigateur . . . 23
4.4 Déployer l'API de MFP . . . 23
4.4.a Création de la stack . . . 23
4.4.b Test avec Bruno . . . 24

TD 4 : Secrets et configs Docker Swarm 25

Prérequis : relancer le cluster et passer en HTTPS . . . 25
0.1 Relancer le cluster . . . 25
0.2 Créer le certificat . . . 25
0.3 Mettre le certificat dans un secret et des configs . . . 26
0.4 Passer Traefik en HTTPS . . . 27
0.5 Relancer Portainer et ajouter les adresses . . . 28
Exercice 1 : Déployer des logiciels . . . 28
1.1 Dozzle avec un utilisateur simple . . . 28
1.1.a Générer users.yml . . . 28
1.1.b Créer le secret dans Portainer . . . 28
1.1.c Déployer Dozzle . . . 29
1.2 Authelia et connexion OIDC à Dozzle . . . 29
1.2.a Créer les clés d'Authelia . . . 29
1.2.b Créer le compte dans un secret . . . 30
1.2.c Créer le client OIDC de Dozzle . . . 30
1.2.d Déployer Authelia . . . 30
1.2.e Brancher Dozzle sur Authelia . . . 31
1.2.f Se connecter à Dozzle par Authelia . . . 32
1.3 Bonus : Outline . . . 33
1.3.a Créer les secrets d'Outline . . . 33
1.3.b Ajouter le client dans Authelia . . . 33
1.3.c Déployer Outline . . . 34
1.3.d Se connecter à Outline par Authelia . . . 35

TD 1 : Dockerisation de My Favorite Places

Exercice 1 : Dockerisation du projet

1.1 Tester le serveur avec yarn dev et Bruno

Le serveur ne démarre pas sans PostgreSQL, j'ai donc lancé une base avant `yarn dev`, avec le mot de passe de `server/src/datasource.ts`.

- `docker run -d --name mfp-db -e POSTGRES_PASSWORD=supersecret -p 5432:5432 postgres:17`
- `docker ps`
- `cd server`
- `yarn install`
- `yarn dev`

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/server$ yarn dev
yarn run v1.22.22
$ nodemon src/index.ts
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: ts,json
[nodemon] starting `ts-node src/index.ts`
⌛ starting server!
query: SELECT version()
query: SELECT * FROM current_schema()
query: START TRANSACTION
query: SELECT * FROM current_schema()
query: SELECT * FROM current_database()
query: SELECT "table_schema", "table_name", obj_description(('"' || "table_schema" || '"."' || "table_name" || '"')::regclass, 'pg_class') AS table_comment
FROM "information_schema"."tables" WHERE ("table_schema" = 'public' AND "table_name" = 'address') OR ("table_schema" = 'public' AND "table_name" = 'user')
query: SELECT * FROM "information_schema"."tables" WHERE "table_schema" = 'public' AND "table_name" = 'typeorm_metadata'
query: CREATE TABLE "address" ("id" SERIAL NOT NULL, "name" character varying NOT NULL, "description" character varying, "lng" double precision NOT NULL, "l
at" double precision NOT NULL, "createdAt" TIMESTAMP NOT NULL DEFAULT now(), "userId" integer, CONSTRAINT "PK_d92de1f82754668b5f5f5dd4fd5" PRIMARY KEY ("id"
))
query: CREATE TABLE "user" ("id" SERIAL NOT NULL, "email" character varying NOT NULL, "hashedPassword" character varying NOT NULL, "createdAt" TIMESTAMP NOT
 NULL DEFAULT now(), CONSTRAINT "PK_cace4a159ff9f2512dd42373760" PRIMARY KEY ("id"))
query: ALTER TABLE "address" ADD CONSTRAINT "FK_d25f1ea79e282cc8a42bd616aa3" FOREIGN KEY ("userId") REFERENCES "user"("id") ON DELETE NO ACTION ON UPDATE NO
 ACTION
query: COMMIT
✅ connected to PgSQL db
🚀 server started on port 3000
```

Figure 1.1. `yarn dev` se connecte à la base et démarre le serveur sur le port 3000.

Dans Bruno, j'ai lancé les requêtes de création d'utilisateur, de connexion, puis de profil.

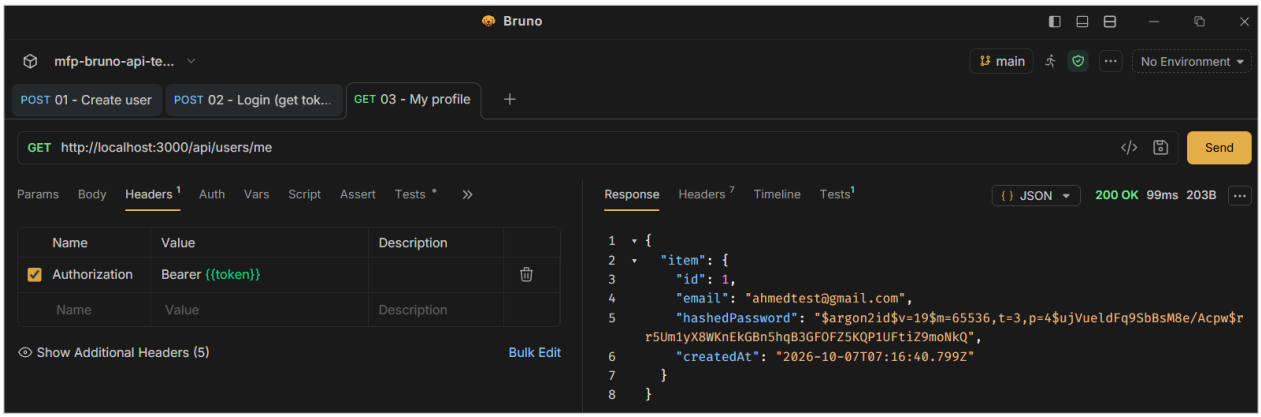


Figure 1.2. `GET /api/users/me` avec le token de la connexion renvoie mon profil.

1.2 Dockeriser le serveur

```
FROM node:24-alpine3.24
WORKDIR /app
COPY package.json yarn.lock ./
RUN yarn install --frozen-lockfile
COPY tsconfig.json ./
COPY src src
RUN yarn build
EXPOSE 3000
CMD ["yarn", "start"]
```

Dans le conteneur, `localhost` ne mène plus à la base. J'ai remplacé les valeurs en dur de `datasource.ts` par des variables d'environnement.

```
host: process.env.DB_HOST || "localhost",
username: process.env.DB_USER || "postgres",
password: process.env.DB_PASSWORD || "supersecret",
database: process.env.DB_NAME || "postgres",
```

J'ai recréé la base sur un réseau `mfp-net`, partagé avec le serveur.

- `cd ..`
- `docker rm -f mfp-db`
- `docker network create mfp-net`
- `docker run -d --name mfp-db --network mfp-net -e POSTGRES_PASSWORD=supersecret postgres:17`
- `docker build -t mfp-server ./server`
- `docker run -d --name mfp-server --network mfp-net -e DB_HOST=mfp-db -p 3000:3000 mfp-server`

Problème rencontré : le conteneur du serveur n'a pas démarré (`address already in use`), car le `yarn dev` de 1.1 occupait encore le port 3000. Je l'ai arrêté, puis j'ai supprimé le conteneur et je l'ai relancé.

- `docker rm -f mfp-server`
- `docker run -d --name mfp-server --network mfp-net -e DB_HOST=mfp-db -p 3000:3000 mfp-server`

La base étant neuve, j'ai recréé l'utilisateur dans Bruno avant de me connecter.

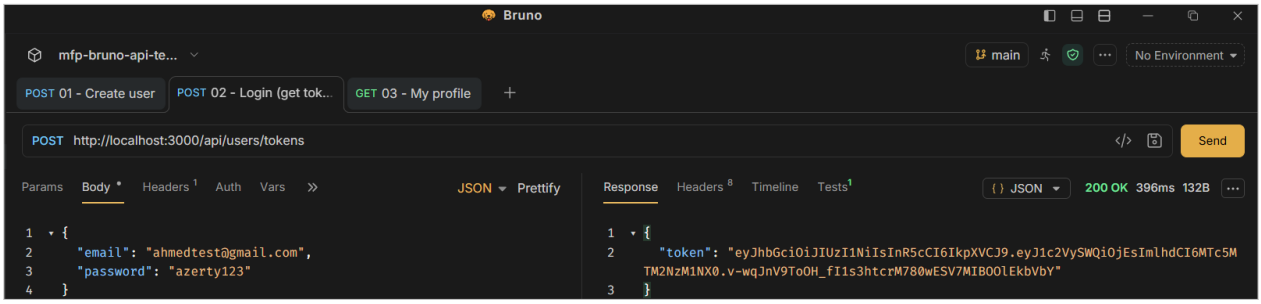


Figure 1.3. La connexion au serveur dans Docker renvoie un token.

- `docker ps`
- `docker logs mfp-server`

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs$ docker ps
CONTAINER ID   IMAGE         COMMAND                  CREATED         STATUS         PORTS
                                NAMES
c1acc99cd601   mfp-server    "docker-entrypoint.s…"   2 minutes ago   Up 2 minutes   0.0.0.
0:3000->3000/tcp, [::]:3000->3000/tcp   mfp-server
c0c978bba581   postgres:17   "docker-entrypoint.s…"   9 minutes ago   Up 9 minutes   5432/t
cp                              mfp-db
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs$ docker logs mfp-server
yarn run v1.22.22
$ node dist/index.js
⏳ starting server!
query: SELECT version()
query: SELECT * FROM current_schema()
query: START TRANSACTION
query: SELECT * FROM current_schema()
query: SELECT * FROM current_database()
query: SELECT "table_schema", "table_name", obj_description(('"' || "table_schema" || '"."'
 || "table_name" || '"')::regclass, 'pg_class') AS table_comment FROM "information_schema".
"tables" WHERE ("table_schema" = 'public' AND "table_name" = 'address') OR ("table_schema"
= 'public' AND "table_name" = 'user')
query: SELECT * FROM "information_schema"."tables" WHERE "table_schema" = 'public' AND "tab
le_name" = 'typeorm_metadata'
query: CREATE TABLE "address" ("id" SERIAL NOT NULL, "name" character varying NOT NULL, "de
scription" character varying, "lng" double precision NOT NULL, "lat" double precision NOT N
ULL, "createdAt" TIMESTAMP NOT NULL DEFAULT now(), "userId" integer, CONSTRAINT "PK_d92de1f
82754668b5f5f5dd4fd5" PRIMARY KEY ("id"))
query: CREATE TABLE "user" ("id" SERIAL NOT NULL, "email" character varying NOT NULL, "hash
edPassword" character varying NOT NULL, "createdAt" TIMESTAMP NOT NULL DEFAULT now(), CONST
RAINT "PK_cace4a159ff9f2512dd42373760" PRIMARY KEY ("id"))
query: ALTER TABLE "address" ADD CONSTRAINT "FK_d25f1ea79e282cc8a42bd616aa3" FOREIGN KEY ("
userId") REFERENCES "user"("id") ON DELETE NO ACTION ON UPDATE NO ACTION
query: COMMIT
✅ connected to PgSQL db
🚀 server started on port 3000
query: START TRANSACTION
query: INSERT INTO "user"("email", "hashedPassword", "createdAt") VALUES ($1, $2, DEFAULT)
RETURNING "id", "createdAt" -- PARAMETERS: ["ahmedtest@gmail.com","$argon2id$v=19$m=65536,t
=3,p=4$QederZF6DNgzDmpXATUdJw$ZXH3ixyadbYjLkJdHMXKkZzYNbVv145xfwJgaaiNssw"]
query: COMMIT
query: SELECT "User"."id" AS "User_id", "User"."email" AS "User_email", "User"."hashedPassw
ord" AS "User_hashedPassword", "User"."createdAt" AS "User_createdAt" FROM "user" "User" WH
ERE (("User"."email" = $1)) LIMIT 1 -- PARAMETERS: ["ahmedtest@gmail.com"]
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs$
```

Figure 1.4. Le serveur crée les tables `address` et `user` dans `mfp-db`, démarre, puis reçoit les requêtes de Bruno.

1.3 Créer un compose.yml pour le serveur et la BDD

```yaml
services:
  db:
    image: postgres:17
    # ...
    volumes:
      - db-data:/var/lib/postgresql/data
  server:
    build: ./server
    # ...
    ports:
      - "3000:3000"
```

Je ne l'ai lancé qu'après avoir ajouté le client, à la question 1.4.

1.4 Dockeriser le client et l'ajouter au compose.yml

1.4.a Dockerfile et lancement du client

```
FROM node:24-alpine3.24
WORKDIR /app
COPY package*.json yarn.lock ./
RUN yarn install --frozen-lockfile
# ...
EXPOSE 5173
CMD ["yarn", "dev", "--host"]
```

Comme pour la base, l'adresse vers laquelle Vite fait suivre `/api` passe dans une variable, dans `vite.config.ts`.

```
target: process.env.API_URL || "http://localhost:3000",
```

- `docker build -t mfp-client ./client`
- `docker run -d --name mfp-client --network mfp-net -e API_URL=http://mfp-server:3000 -p 5173:5173 mfp-client`
- `docker ps`
- `docker logs mfp-client`

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs$ docker run -d --name mfp-client --network mfp-net \
  -e API_URL=http://mfp-server:3000 \
  -p 5173:5173 \
  mfp-client
846fcb38043009ec7f4d7a70d92cf27f9d033577d0055c71cda60564b4b47207
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs$ docker ps
CONTAINER ID   IMAGE         COMMAND                  CREATED          STATUS          PORT
S                                       NAMES
846fcb380430   mfp-client    "docker-entrypoint.s…"   19 seconds ago   Up 17 seconds   0.0.
0.0:5173->5173/tcp, [::]:5173->5173/tcp   mfp-client
c1acc99cd601   mfp-server    "docker-entrypoint.s…"   13 minutes ago   Up 13 minutes   0.0.
0.0:3000->3000/tcp, [::]:3000->3000/tcp   mfp-server
c0c978bba581   postgres:17   "docker-entrypoint.s…"   20 minutes ago   Up 20 minutes   5432
/tcp                                    mfp-db
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs$ docker logs mfp-client
yarn run v1.22.22
$ vite --host

  VITE v7.3.1  ready in 798 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: http://172.18.0.4:5173/
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs$
```

Figure 1.5. `mfp-db`, `mfp-server` et `mfp-client` sont `Up`, Vite écoute sur le port 5173.

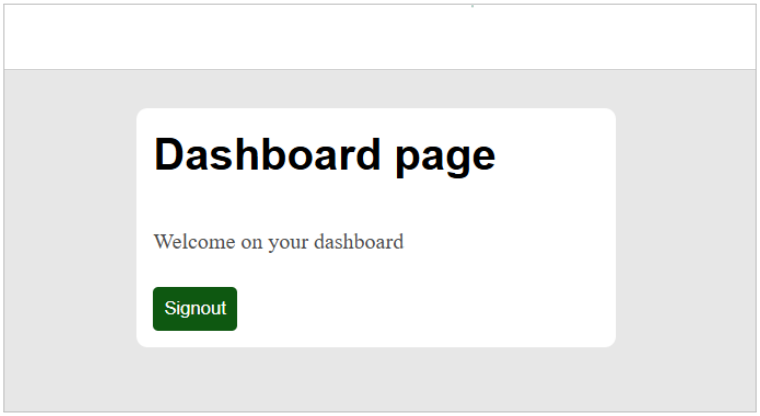


Figure 1.6. Connexion réussie sur `localhost:5173` avec le compte créé dans Bruno.

1.4.b Ajout au compose.yml et lancement

```yaml
client:
  build: ./client
  # ...
  ports:
    - "5173:5173"
  depends_on:
    - server
```

J'ai supprimé les conteneurs lancés à la main, qui occupaient encore les ports 3000 et 5173.

- `docker rm -f mfp-client mfp-server mfp-db`
- `docker network rm mfp-net`
- `docker compose up -d --build`
- `docker compose ps`
- `docker compose logs server`

Problème rencontré : au premier essai, avec un simple `depends_on`, le serveur s'arrêtait sur `ECONNREFUSED`, parce qu'il démarrait avant PostgreSQL. J'ai ajouté le healthcheck de la question 2.1, puis j'ai relancé les trois commandes `docker compose`.

```
[+] up 7/7
 ✔Image mfp-client       Built                                                6.2s
 ✔Image mfp-server       Built                                                6.0s
 ✔Network mfp_default    Created                                              0.1s
 ✔Volume mfp_db-data     Created                                              0.0s
 ✔Container mfp-db-1     Healthy                                              8.3s
 ✔Container mfp-server-1 Started                                              8.6s
 ✔Container mfp-client-1 Started                                              8.9s
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs$ docker compose ps
NAME           IMAGE         COMMAND                  SERVICE   CREATED          STATUS
               PORTS
mfp-client-1   mfp-client    "docker-entrypoint.s…"   client    19 seconds ago   Up 9 secon
ds             0.0.0.0:5173->5173/tcp, [::]:5173->5173/tcp
mfp-db-1       postgres:17   "docker-entrypoint.s…"   db        19 seconds ago   Up 17 seco
nds (healthy)  5432/tcp
mfp-server-1   mfp-server    "docker-entrypoint.s…"   server    19 seconds ago   Up 10 seco
nds            0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
```

Figure 1.7. `mfp-db-1` est `Healthy` avant le démarrage du serveur, et les 3 services sont `Up`.

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs$   docker compose logs server
server-1  | yarn run v1.22.22
server-1  | $ node dist/index.js
server-1  | ⏳ starting server!
server-1  | query: SELECT version()
server-1  | query: SELECT * FROM current_schema()
server-1  | query: START TRANSACTION
server-1  | query: SELECT * FROM current_schema()
server-1  | query: SELECT * FROM current_database()
server-1  | query: SELECT "table_schema", "table_name", obj_description(('"' || "table_sche
ma" || '"."' || "table_name" || '"')::regclass, 'pg_class') AS table_comment FROM "informat
ion_schema"."tables" WHERE ("table_schema" = 'public' AND "table_name" = 'address') OR ("ta
ble_schema" = 'public' AND "table_name" = 'user')
server-1  | query: SELECT * FROM "information_schema"."tables" WHERE "table_schema" = 'publ
ic' AND "table_name" = 'typeorm_metadata'
server-1  | query: CREATE TABLE "address" ("id" SERIAL NOT NULL, "name" character varying N
OT NULL, "description" character varying, "lng" double precision NOT NULL, "lat" double pre
cision NOT NULL, "createdAt" TIMESTAMP NOT NULL DEFAULT now(), "userId" integer, CONSTRAINT
 "PK_d92de1f82754668b5f5f5dd4fd5" PRIMARY KEY ("id"))
server-1  | query: CREATE TABLE "user" ("id" SERIAL NOT NULL, "email" character varying NOT
 NULL, "hashedPassword" character varying NOT NULL, "createdAt" TIMESTAMP NOT NULL DEFAULT
now(), CONSTRAINT "PK_cace4a159ff9f2512dd42373760" PRIMARY KEY ("id"))
server-1  | query: ALTER TABLE "address" ADD CONSTRAINT "FK_d25f1ea79e282cc8a42bd616aa3" FO
REIGN KEY ("userId") REFERENCES "user"("id") ON DELETE NO ACTION ON UPDATE NO ACTION
server-1  | query: COMMIT
server-1  | ✅ connected to PgSQL db
server-1  | 🚀 server started on port 3000
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs$
```

Figure 1.8. Le serveur se connecte à la base, crée les tables, puis démarre sur le port 3000.

Le volume étant neuf, j'ai recréé l'utilisateur avec Bruno, et la connexion sur le port 5173 a marché. Ensuite, j'ai mis le nom `db` en dur dans `datasource.ts`, à la place des variables.

- `docker compose down`

1.5 Communication entre la BDD, le serveur et le front

- BDD et serveur : même réseau `mfp_default` (créé par Compose), le serveur appelle `db:5432`.
- Serveur et front : le navigateur appelle `localhost:5173/api`, et le conteneur `client` transmet à `server:3000`.

Exercice 2 : Démarrage, redémarrage et images légères

2.1 Démarrer le serveur une fois la BDD prête

Ajouté pendant la question 1.4, après le plantage du serveur.

```yaml
db:
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 2s
    # ...
server:
  depends_on:
    db:
      condition: service_healthy
```

2.2 Redémarrer automatiquement les services

J'ai mis cette ligne sur les trois services.

```
restart: unless-stopped
```

Pour vérifier, j'ai arrêté le serveur moi-même de deux façons. Quand j'ai tué son processus, Docker l'a relancé tout seul, mais après un arrêt manuel du conteneur, il est resté arrêté.

2.3 Optimiser les images avec les build stages

2.3.a Serveur

```
FROM node:24-alpine3.24 AS deps
# ...
FROM node:24-alpine3.24 AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
# ...
FROM node:24-alpine3.24 AS main
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY package.json yarn.lock ./
RUN yarn install --frozen-lockfile --production && yarn cache clean
# ...
CMD ["yarn", "start"]
```

2.3.b Client

Les étapes `deps` et `builder` sont les mêmes. La dernière sert le build avec nginx, qui fait aussi suivre `/api` au serveur.

```
# ...
FROM nginx:1.29-alpine AS main
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=builder /app/dist /usr/share/nginx/html
EXPOSE 5173
```

```
server {
    listen 5173;
    # ...
    location / {
        try_files $uri $uri/ /index.html;
    }
    location /api/ {
        proxy_pass http://server:3000;
    }
}
```

2.3.c Résultat

- `docker compose up -d --build`
- `docker images`

Dans `docker images`, `mfp-server` passe de 477 Mo à 337 Mo, soit 140 Mo de moins, et `mfp-client` de 1,49 Go à 93 Mo.

TD 2 : Cluster Docker Swarm

Exercice 1 : DinD et DooD

- DinD : un daemon Docker tourne dans le conteneur (image `docker:dind`, lancée en `privileged`).
- DooD : le conteneur utilise le daemon de l'hôte, par le socket `/var/run/docker.sock` monté en volume.
- Pour tester Swarm, j'ai retenu DinD : un `compose.yml` avec 4 conteneurs DinD, 1 manager et 3 nœuds, qui jouent chacun une machine du cluster (exercice 2).

Prérequis : images sur un registry

Deux workflows GitHub Actions, `.github/workflows/build.server.yml` et `build.client.yml`, construisent les images du serveur et du client quand un push sur `main` modifie leur code, puis les publient sur GHCR.

- `git push`

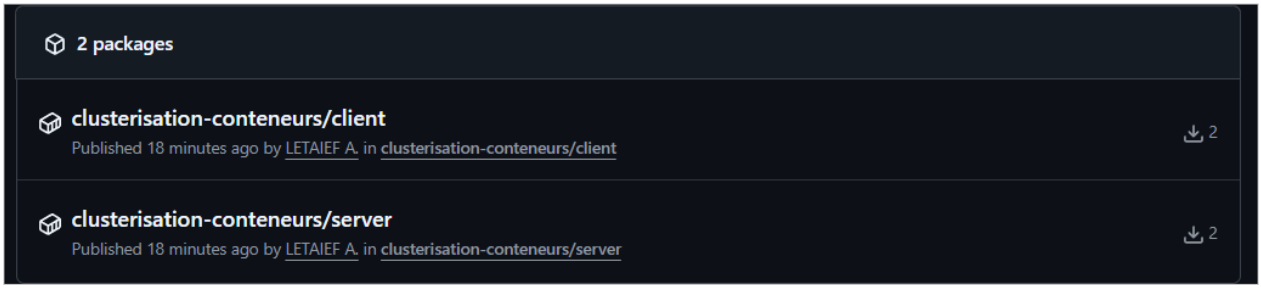


Figure 2.1. Les images `server` et `client` sont publiées sur GHCR.

Exercice 2 : Création du cluster Docker Swarm

2.1 Créer et lancer le manager et les 3 nœuds DinD

Dans `swarm/compose.yml`, j'ai déclaré un conteneur `manager` et trois conteneurs `node1`, `node2` et `node3`, tous en `docker:dind` avec `privileged: true`.

```yaml
services:
  manager:
    image: docker:dind
    hostname: manager
    privileged: true
  node1:
    image: docker:dind
    hostname: node1
    privileged: true
  # node2 et node3 : pareil que node1
```

- `cd swarm`
- `docker compose up -d`
- `docker ps`

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs/swarm$  docker compose up -d
[+] up 22/22
 ✔Image docker:dind          Pulled                                                  29.5s
 ✔Network swarm_default      Created                                                  0.1s
 ✔Container swarm-node3-1    Started                                                  1.4s
 ✔Container swarm-manager-1  Started                                                  1.5s
 ✔Container swarm-node1-1    Started                                                  1.4s
 ✔Container swarm-node2-1    Started                                                  1.5s
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs/swarm$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED             STATUS
       PORTS                                                 NAMES
8db993eaa83c   docker:dind    "dockerd-entrypoint.…"   24 seconds ago      Up 22 seconds
       2375-2376/tcp                                         swarm-manager-1
12a3a3bf1b09   docker:dind    "dockerd-entrypoint.…"   24 seconds ago      Up 22 seconds
       2375-2376/tcp                                         swarm-node2-1
aacec484287b   docker:dind    "dockerd-entrypoint.…"   24 seconds ago      Up 22 seconds
       2375-2376/tcp                                         swarm-node3-1
57aa55d79f91   docker:dind    "dockerd-entrypoint.…"   24 seconds ago      Up 22 seconds
       2375-2376/tcp                                         swarm-node1-1
11ed06c1a8a4   mfp-client     "/docker-entrypoint.…"   About an hour ago   Up About an hour
       80/tcp, 0.0.0.0:5173->5173/tcp, [::]:5173->5173/tcp   mfp-client-1
f6b1980ab4cb   c9ab01022362   "docker-entrypoint.s…"   2 hours ago         Up About an hour
       0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp           mfp-server-1
4c4030c6d31d   postgres:17    "docker-entrypoint.s…"   2 hours ago         Up 2 hours (heal
thy)   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp           mfp-db-1
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
```

Figure 2.2. Les 4 conteneurs `manager`, `node1`, `node2` et `node3` sont `Up`.

- `docker compose exec manager docker ps`
- `docker compose exec node1 docker ps`

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs/swarm$ docker compose exec manager docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs/swarm$ docker compose exec node1 docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs/swarm$
```

Figure 2.3. `docker ps` répond dans les conteneurs, avec une liste vide.

2.2 Initialiser le cluster depuis le manager

- `docker compose exec manager docker swarm init`

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs/swarm$  docker compose exec manager docker swarm init
Swarm initialized: current node (ut2j0ctz6094h5y236qwnbii5) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-53jydxiofa3tv6px9x9q02zwdnobcv5ql96inowzt6hb8retjj-3
kr9urlpy2q0j7wjytrs1pm19 172.19.0.5:2377
```

Figure 2.4. Le conteneur `manager` devient le manager du swarm et affiche la commande `docker swarm join`, avec le token des workers.

2.3 Faire rejoindre les nœuds comme workers

J'ai mis le token dans une variable, et les nœuds rejoignent `manager:2377`, le nom du service.

- `TOKEN=$(docker compose exec manager docker swarm join-token -q worker)`
- `docker compose exec node1 docker swarm join --token $TOKEN manager:2377`
- `docker compose exec node2 docker swarm join --token $TOKEN manager:2377`
- `docker compose exec node3 docker swarm join --token $TOKEN manager:2377`

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusteris
ation-Conteneurs/swarm$  docker compose exec node1 docker swarm join --token $TOKEN manager:2377
This node joined a swarm as a worker.
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusteris
ation-Conteneurs/swarm$  docker compose exec node2 docker swarm join --token $TOKEN manager:2377
This node joined a swarm as a worker.
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusteris
ation-Conteneurs/swarm$  docker compose exec node3 docker swarm join --token $TOKEN manager:2377
This node joined a swarm as a worker.
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusteris
ation-Conteneurs/swarm$
```

Figure 2.5. Les 3 nœuds répondent `This node joined a swarm as a worker.`

2.4 Lister les nœuds depuis le manager

- `docker compose exec manager ash`
- `docker node ls`
- `exit`

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusteris
ation-Conteneurs/swarm$  docker compose exec manager ash
/ #  docker node ls
ID                            HOSTNAME   STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
ut2j0ctz6094h5y236qwnbii5 *   manager    Ready     Active         Leader           29.8.2
gkqoimullj55ae70vubvjpq3t     node1      Ready     Active                          29.8.2
htlyy0s49t7h787sznf9syr0h     node2      Ready     Active                          29.8.2
tih4g88m7eipgra6a02nwvi17     node3      Ready     Active                          29.8.2
/ # exit
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusteris
ation-Conteneurs/swarm$
```

Figure 2.6. Le cluster compte 4 nœuds `Ready` : le manager (`Leader`) et les 3 workers `node1` à `node3`.

Exercice 3 : Tests du cluster

3.1 Créer la stack hello-world

```yaml
services:
  hello:
    image: hello-world
    deploy:
      replicas: 2
```

3.2 Copier la stack dans le manager

J'ai ajouté le volume anonyme sur `/home/manager` dans `swarm/compose.yml`.

```yaml
manager:
  # ...
  volumes:
    - /home/manager
```

- `docker compose up -d manager`
- `docker compose exec manager docker node ls`
- `docker compose exec manager mkdir -p /home/manager`
- `docker compose cp hello-world.compose.yml manager:/home/manager/`
- `docker compose exec manager ls /home/manager`

Le manager a été recréé, et `docker node ls` affiche toujours le manager et les 3 workers en `Ready`. `ls` affiche `hello-world.compose.yml`.

3.3 Déployer la stack et observer les nœuds

Problème rencontré : après un redémarrage du PC, les nœuds restaient `Down`, car Swarm avait gardé l'ancienne IP du manager. J'ai fait quitter le cluster à toutes les machines et je l'ai recréé.

- `docker compose start`
- `docker compose exec manager docker node ls`
- `docker compose exec node1 docker swarm leave --force`
- `docker compose exec node2 docker swarm leave --force`
- `docker compose exec node3 docker swarm leave --force`
- `docker compose exec manager docker swarm leave --force`
- `docker compose exec manager docker swarm init`
- `TOKEN=$(docker compose exec manager docker swarm join-token -q worker)`
- `docker compose exec node1 docker swarm join --token $TOKEN manager:2377`
- `docker compose exec node2 docker swarm join --token $TOKEN manager:2377`
- `docker compose exec node3 docker swarm join --token $TOKEN manager:2377`
- `docker compose exec manager docker node ls`

Le manager et les 3 workers sont de nouveau `Ready`.

- `docker compose exec manager docker stack deploy -c /home/manager/hello-world.compose.yml hello`
- `docker compose exec manager docker service ls`
- `docker compose exec manager docker service ps hello_hello`

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec manager docker stack deploy -c /home/manager/hello-world.compose.yml hello
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network hello_default
Creating service hello_hello
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec manager docker service ls
ID             NAME          MODE         REPLICAS   IMAGE                 PORTS
8f2u4qupmve5   hello_hello   replicated   0/2        hello-world:latest
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec manager docker service ps hello_hello
ID             NAME                IMAGE                 NODE      DESIRED STATE   CURRENT STATE                ERROR     PORTS
k5hvj9cbzhag   hello_hello.1       hello-world:latest    node3     Ready           Ready less than a second ago
4efsfp7ey48b    \_ hello_hello.1   hello-world:latest    manager   Shutdown        Complete 1 second ago
kzz1gxv19iyc    \_ hello_hello.1   hello-world:latest    node1     Shutdown        Complete 7 seconds ago
pouc24vc250r    \_ hello_hello.1   hello-world:latest    node1     Shutdown        Complete 14 seconds ago
9e3au9hbifkm    \_ hello_hello.1   hello-world:latest    node2     Shutdown        Complete 20 seconds ago
j7s4gs6stm7r   hello_hello.2       hello-world:latest    node2     Ready           Ready 3 seconds ago
jr9fnrkp9521    \_ hello_hello.2   hello-world:latest    node3     Shutdown        Complete 4 seconds ago
uufrb0j0lc01    \_ hello_hello.2   hello-world:latest    manager   Shutdown        Complete 10 seconds ago
x8sj2zbnbq2j    \_ hello_hello.2   hello-world:latest    manager   Shutdown        Complete 17 seconds ago
fbzxjugknqqu    \_ hello_hello.2   hello-world:latest    node1     Shutdown        Complete 23 seconds ago
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
```

Figure 2.7. `hello_hello` reste à `0/2`, et ses tâches passent d'un nœud à l'autre.

Pour comparer, un service nginx qui reste allumé affiche bien `3/3`.

- `docker compose exec manager docker service create --name web --replicas 3 nginx:alpine`
- `docker compose exec manager docker service ls`
- `docker compose exec manager docker service rm web`
- `docker compose exec manager docker ps -a`
- `docker compose exec node1 docker ps -a`
- `docker compose exec node2 docker ps -a`
- `docker compose exec node3 docker ps -a`

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$  docker compose exec manager do
cker ps -a
CONTAINER ID   IMAGE                 COMMAND    CREATED          STATUS                       PORTS     NAMES
f355638693eb   hello-world:latest    "/hello"   9 seconds ago    Exited (0) 2 seconds ago             hello_hello.1.xsi3ohviq3j43lx5xuadtdrvw
f24aef73124b   hello-world:latest    "/hello"   16 seconds ago   Exited (0) 10 seconds ago            hello_hello.1.7sigjlu29gbypa3hod40cx39m
bfbf0de6f4ad   hello-world:latest    "/hello"   24 seconds ago   Exited (0) 17 seconds ago            hello_hello.2.y2xt2fd35e37cbadcod13i7pa
0833d06d9755   hello-world:latest    "/hello"   31 seconds ago   Exited (0) 24 seconds ago            hello_hello.2.5a31wbqdf4gqphyevylmc2vky
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec node1 docke
r ps -a
CONTAINER ID   IMAGE                 COMMAND    CREATED          STATUS                       PORTS     NAMES
b12bf34dff67   hello-world:latest    "/hello"   2 seconds ago    Created                              hello_hello.2.noviml5uf5s40265yxrk5806n
07d1bfb7b503   hello-world:latest    "/hello"   27 seconds ago   Exited (0) 22 seconds ago            hello_hello.2.6uho387zyzllzoeie4jiux84v
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec node2 docke
r ps -a
CONTAINER ID   IMAGE                 COMMAND    CREATED          STATUS                       PORTS     NAMES
939973e82e41   hello-world:latest    "/hello"   22 seconds ago   Exited (0) 16 seconds ago            hello_hello.1.0fmw0lejau2izpjzruozpsw7v
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec node3 docke
r ps -a
CONTAINER ID   IMAGE                 COMMAND    CREATED          STATUS                       PORTS     NAMES
ada9e02e6213   hello-world:latest    "/hello"   23 seconds ago   Exited (0) 17 seconds ago            hello_hello.1.xg3wkzwk1r2udwnx33bxxaqxb
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
```

Figure 2.8. Chaque nœud, manager compris, a ses propres conteneurs `hello-world` en `Exited (0)`.

J'en déduis que Swarm répartit les réplicas sur tous les nœuds, manager compris, et que `docker ps` ne montre que les conteneurs du nœud où on le lance. `hello-world` s'arrête juste après son message, alors Swarm le relance en boucle : d'où le `0/2`, et `docker ps` sans `-a` n'affiche presque rien.

3.4 Faire varier la clause deploy

3.4.a Sur le manager seulement (par rôle)

```yaml
deploy:
  replicas: 2
  placement:
    constraints:
      - node.role == manager
```

- `docker compose exec manager docker stack rm hello`
- `docker compose cp hello-world-manager.compose.yml manager:/home/manager/`
- `docker compose exec manager docker stack deploy -c /home/manager/hello-world-manager.compose.yml hello`
- `docker compose exec manager docker service ps hello_hello`

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
 docker compose exec manager docker service ps hello_hello
ID             NAME                IMAGE                NODE      DESIRED STATE   CURRENT STATE                ERROR
 PORTS
zfjhmrzuo4fz   hello_hello.1       hello-world:latest   manager   Ready           Ready less than a second ago

0z7vmg3rvvnu    \_ hello_hello.1   hello-world:latest   manager   Shutdown        Complete 1 second ago

0s8l81lgrk1a    \_ hello_hello.1   hello-world:latest   manager   Shutdown        Complete 7 seconds ago

qkg5g80puhls   hello_hello.2       hello-world:latest   manager   Ready           Ready less than a second ago

lod0t1rgd1ya    \_ hello_hello.2   hello-world:latest   manager   Shutdown        Complete 1 second ago

vq2ejxa6qsuo    \_ hello_hello.2   hello-world:latest   manager   Shutdown        Complete 7 seconds ago

letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
```

Figure 2.9. Toutes les tâches de `hello_hello` tournent sur le manager.

3.4.b Sur tous les nœuds (mode global)

```
deploy:
  mode: global
```

- `docker compose exec manager docker stack rm hello`
- `docker compose cp hello-world-global.compose.yml manager:/home/manager/`
- `docker compose exec manager docker stack deploy -c /home/manager/hello-world-global.compose.yml hello`
- `docker compose exec manager docker service ls`
- `docker compose exec manager docker service ps hello_hello`

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec manager do
cker service ls
ID             NAME          MODE      REPLICAS   IMAGE              PORTS
rno40lm6b9zl   hello_hello   global    0/0        hello-world:latest
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec manager do
cker service ps hello_hello
ID             NAME                                    IMAGE                NODE      DESIRED STATE   CURRENT STATE                ERROR     PORTS
p16kq27gcitt   hello_hello.6f2nkebkucu24f1jdpleo8jkg    hello-world:latest   node2     Ready           Ready less than a second ago
spzm3npdcc0n    \_ hello_hello.6f2nkebkucu24f1jdpleo8jkg   hello-world:latest   node2     Shutdown        Complete 1 second ago
fbfe974s5a7d    \_ hello_hello.6f2nkebkucu24f1jdpleo8jkg   hello-world:latest   node2     Shutdown        Complete 9 seconds ago
kfi1wj4o3psc    \_ hello_hello.6f2nkebkucu24f1jdpleo8jkg   hello-world:latest   node2     Shutdown        Complete 15 seconds ago
ozlzlcgou9mc    \_ hello_hello.6f2nkebkucu24f1jdpleo8jkg   hello-world:latest   node2     Shutdown        Complete 22 seconds ago
74fck10ko4cn   hello_hello.6o2far8qyljljgwahds6pqrgb    hello-world:latest   node3     Ready           Ready less than a second ago
wgp9hf0ivajv    \_ hello_hello.6o2far8qyljljgwahds6pqrgb   hello-world:latest   node3     Shutdown        Complete 1 second ago
3274m4l4cf1d    \_ hello_hello.6o2far8qyljljgwahds6pqrgb   hello-world:latest   node3     Shutdown        Complete 9 seconds ago
c8lp7t71y9x8    \_ hello_hello.6o2far8qyljljgwahds6pqrgb   hello-world:latest   node3     Shutdown        Complete 15 seconds ago
68a30dpoer2i    \_ hello_hello.6o2far8qyljljgwahds6pqrgb   hello-world:latest   node3     Shutdown        Complete 22 seconds ago
r7cz5gaxghs8   hello_hello.kv0o3q4vy9cttsv66slml7q2n    hello-world:latest   node1     Ready           Ready 3 seconds ago
uipommkun2bz    \_ hello_hello.kv0o3q4vy9cttsv66slml7q2n   hello-world:latest   node1     Shutdown        Complete 4 seconds ago
iuw7w43mt34s    \_ hello_hello.kv0o3q4vy9cttsv66slml7q2n   hello-world:latest   node1     Shutdown        Complete 11 seconds ago
md9jqa42g20l    \_ hello_hello.kv0o3q4vy9cttsv66slml7q2n   hello-world:latest   node1     Shutdown        Complete 17 seconds ago
ntjsnf5aqd2t    \_ hello_hello.kv0o3q4vy9cttsv66slml7q2n   hello-world:latest   node1     Shutdown        Complete 23 seconds ago
vrwzcp2e75xy   hello_hello.vktrun1l2rvmov829r3pkq9fr    hello-world:latest   manager   Ready           Ready less than a second ago
t9dwf3qketly    \_ hello_hello.vktrun1l2rvmov829r3pkq9fr   hello-world:latest   manager   Shutdown        Complete 2 seconds ago
ok3tbu0hd9zo    \_ hello_hello.vktrun1l2rvmov829r3pkq9fr   hello-world:latest   manager   Shutdown        Complete 9 seconds ago
rtsx0d3rbglq    \_ hello_hello.vktrun1l2rvmov829r3pkq9fr   hello-world:latest   manager   Shutdown        Complete 15 seconds ago
d5puswsuv948    \_ hello_hello.vktrun1l2rvmov829r3pkq9fr   hello-world:latest   manager   Shutdown        Complete 22 seconds ago
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
```

Figure 2.10. En mode `global`, une tâche par nœud : `node1`, `node2`, `node3` et le manager.

TD 3 : Traefik et Portainer

Exercice 1 : Déployer Traefik

1.1 Relancer le cluster

J'ai publié le port 80 du manager dans `mfp/swarm/compose.yml`, pour la question 1.3.

```yaml
manager:
  # ...
  ports:
    - "80:80"
```

- `cd mfp/swarm`
- `docker compose up -d`
- `docker compose exec manager docker node ls`

Seul le manager a été recréé, `docker node ls` affiche toujours le manager et les 3 workers en `Ready`.

1.2 Déployer la stack Traefik sur le manager

J'ai créé le réseau `web` en overlay, supprimé la stack `hello` du TD 2, puis déployé la stack du cours (`traefik.compose.yml`).

- `docker compose exec manager docker network create --driver overlay web`
- `docker compose exec manager docker network ls --filter name=web`
- `docker compose exec manager docker stack rm hello`
- `docker compose cp traefik.compose.yml manager:/home/manager/`
- `docker compose exec manager docker stack deploy -c /home/manager/traefik.compose.yml traefik`
- `docker compose exec manager docker service ls`

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose cp traefik.compose.yml manager:/home/manager/
 swarm-manager-1 Copying traefik.compose.yml to swarm-manager-1:/home/manager/
 swarm-manager-1 Copied traefik.compose.yml to swarm-manager-1:/home/manager/
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec manager docker stack deploy -c /home/manage
r/traefik.compose.yml traefik
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating service traefik_traefik
Creating service traefik_whoami
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec manager docker service ls
ID             NAME             MODE         REPLICAS   IMAGE                  PORTS
34nw81r3tlo8   traefik_traefik   replicated   1/1        traefik:v3.6           *:80->80/tcp
mbp8wvpskz9n   traefik_whoami    replicated   1/1        traefik/whoami:latest
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
```

Figure 3.1. `traefik_traefik` et `traefik_whoami` tournent, seul Traefik publie un port.

1.3 Accéder à Traefik depuis le navigateur

J'ai ajouté ces lignes au fichier hosts de Windows (`C:\Windows\System32\drivers\etc\hosts`), ouvert en administrateur.

```
127.0.0.1   traefik.swarm.localhost
127.0.0.1   whoami.swarm.localhost
```

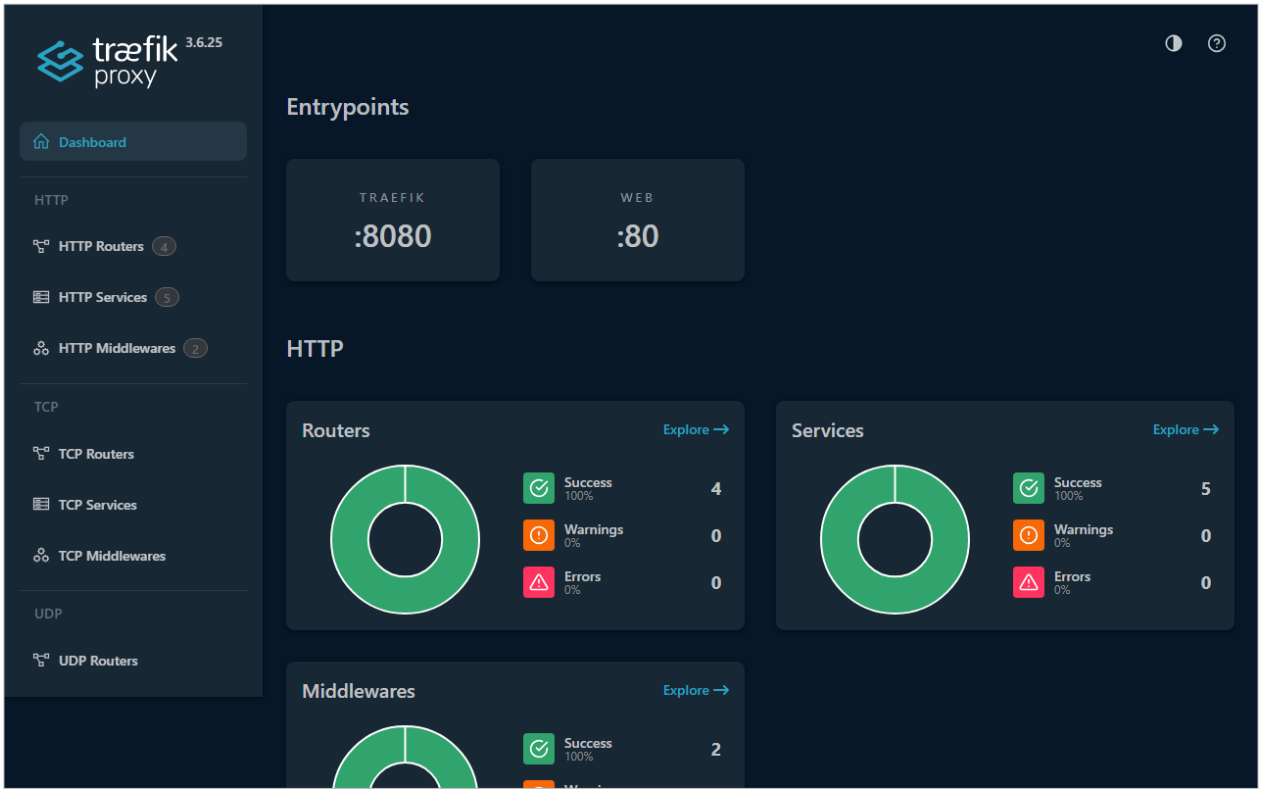


Figure 3.2. Le tableau de bord de Traefik sur `traefik.swarm.localhost`.

```
Hostname: e96fb8cd89f8
IP: 127.0.0.1
IP: ::1
IP: 10.0.2.6
IP: 172.18.0.3
RemoteAddr: 10.0.2.3:34768
GET / HTTP/1.1
Host: whoami.swarm.localhost
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
HeadlessChrome/156.0.0.0 Safari/537.36 Edg/156.0.0.0
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/
signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: fr,fr-FR;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6
Sec-Ch-Ua: "Not:A-Brand";v="8", "Chromium";v="156", "Microsoft Edge";v="156"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: none
Sec-Fetch-User: ?1
Upgrade-Insecure-Requests: 1
X-Forwarded-For: 10.0.0.2
X-Forwarded-Host: whoami.swarm.localhost
X-Forwarded-Port: 80
X-Forwarded-Proto: http
X-Forwarded-Server: 62ea62114d18
X-Real-Ip: 10.0.0.2
```

Figure 3.3. `whoami.swarm.localhost` répond, avec les en-têtes `X-Forwarded-*` ajoutés par Traefik.

Exercice 2 : Déployer une autre application

2.1 Tester l'application avec Docker Compose

- `cd ../../voting-app`
- `docker compose up -d`

Problème rencontré : le téléchargement des images a échoué avec `429 Too Many Requests`, la limite de Docker Hub sans compte. Je me suis connecté avec `docker login`, puis j'ai relancé.

- `docker login`
- `docker compose up -d`
- `docker compose ps`
- `curl -X POST -d vote=a http://localhost:8080/` (3 fois)
- `curl -X POST -d vote=b http://localhost:8080/` (2 fois)

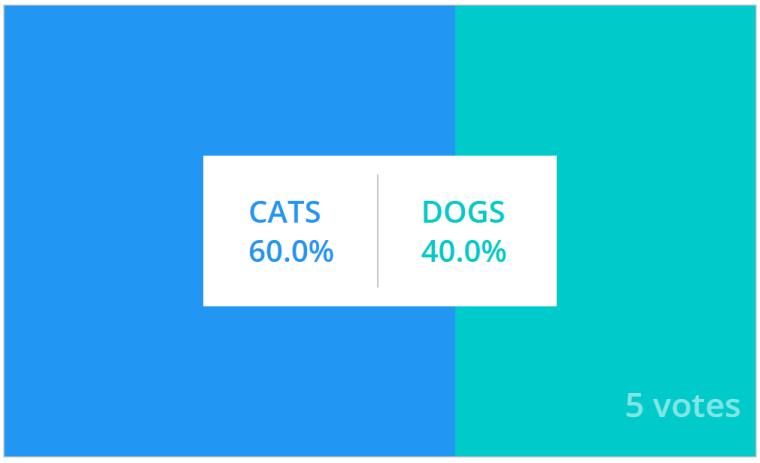


Figure 3.4. Les 5 votes envoyés (3 Cats, 2 Dogs) apparaissent sur la page des résultats.

- `docker compose down`

2.2 Déployer l'application sur le cluster Swarm

Je me suis connecté à Docker Hub dans le manager, et `--with-registry-auth` transmet cette connexion aux nœuds.

- `cd ../mfp/swarm`
- `docker compose exec manager docker login`
- `docker compose cp ../../voting-app/docker-stack.yml manager:/home/manager/`
- `docker compose exec manager docker stack deploy --with-registry-auth -c /home/manager/docker-stack.yml voting`
- `docker compose exec manager docker stack services voting`

Au bout d'une minute, les 5 services sont en ligne, mais les ports 8080 et 8081 ne sont pas joignables depuis mon PC.

2.3 Publier le vote et les résultats avec Traefik

Dans une copie de la stack, `docker-stack.traefik.yml`, j'ai remplacé les `ports` de `vote` et `result` par des labels Traefik, sur le modèle de `whoami`. Voici la partie `vote`, `result` suit le même modèle.

```
networks:
  - frontend
  - web
deploy:
  replicas: 2
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.vote.rule=Host(`vote.swarm.localhost`)"
    - "traefik.http.routers.vote.entrypoints=web"
    - "traefik.http.services.vote.loadbalancer.server.port=80"
```

- `docker compose cp ../../voting-app/docker-stack.traefik.yml manager:/home/manager/`
- `docker compose exec manager docker stack deploy --with-registry-auth -c /home/manager/docker-stack.traefik.yml voting`
- `docker compose exec manager docker stack services voting`

```
127.0.0.1   vote.swarm.localhost
127.0.0.1   result.swarm.localhost
```

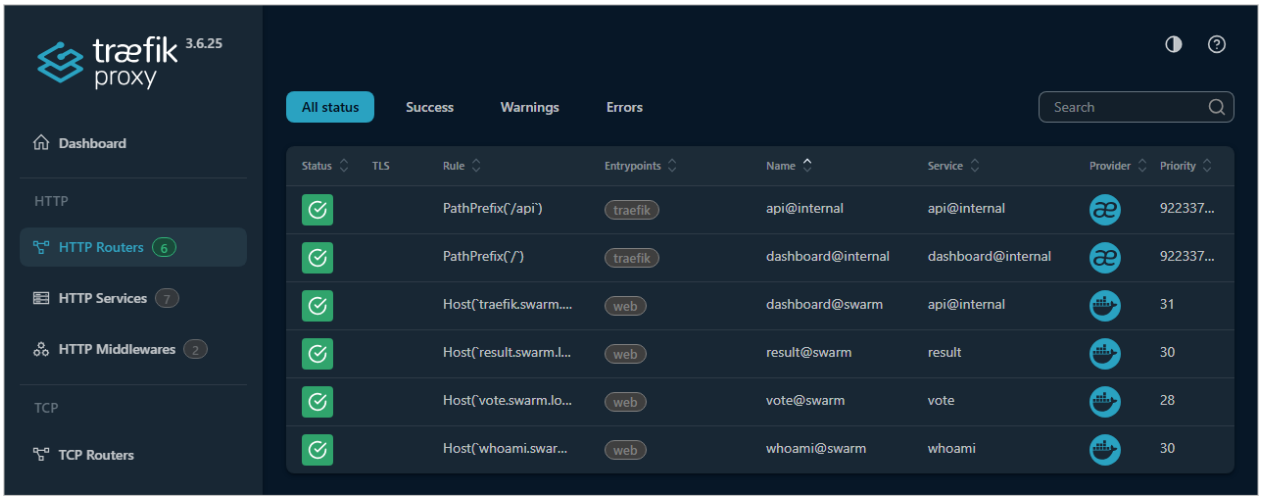


Figure 3.5. Les routeurs `vote@swarm` et `result@swarm` dans Traefik.

- `curl -X POST -d vote=a http://vote.swarm.localhost/` (2 fois)
- `curl -X POST -d vote=b http://vote.swarm.localhost/` (3 fois)

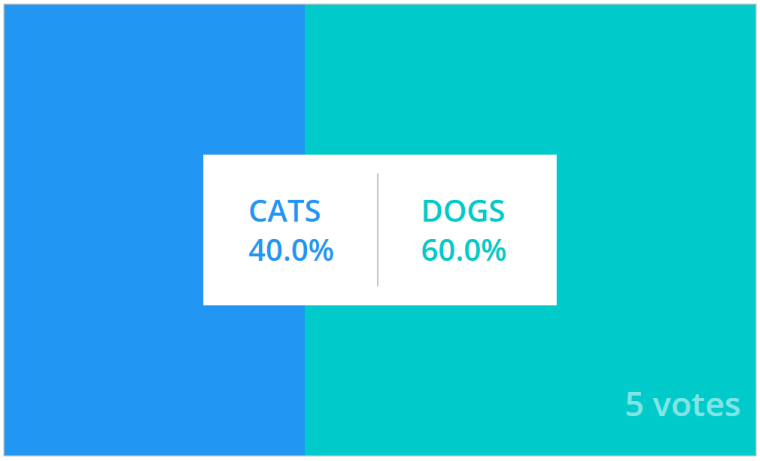


Figure 3.6. `result.swarm.localhost` affiche les 5 votes envoyés au cluster (2 Cats, 3 Dogs).

Exercice 3 : Déployer Portainer

3.1 Déployer Portainer sur le cluster

J'ai utilisé la stack de la documentation de Portainer pour Swarm.

- `curl -sL https://downloads.portainer.io/ce-lts/portainer-agent-stack.yml -o portainer-agent-stack.yml`
- `docker compose cp portainer-agent-stack.yml manager:/home/manager/`
- `docker compose exec manager docker stack deploy --with-registry-auth -c /home/manager/portainer-agent-stack.yml portainer`
- `docker compose exec manager docker stack services portainer`
- `docker compose exec manager docker ps --filter name=portainer --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}'`

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ curl -sL https://downloads.portainer.io/ce-lts/portainer-agent-stack.yml -o portainer-agent-stack.yml
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose cp portainer-agent-stack.yml manager:/home/manager/
 swarm-manager-1 Copying portainer-agent-stack.yml to swarm-manager-1:/home/manager/
 swarm-manager-1 Copied portainer-agent-stack.yml to swarm-manager-1:/home/manager/
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose exec manager docker stack deploy --with-registry-auth -c /home/manager/portainer-agent-stack.yml portainer
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network portainer_agent_network
Creating service portainer_agent
Creating service portainer_portainer
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose exec manager docker stack services portainer
ID             NAME                MODE         REPLICAS   IMAGE                          PORTS
gdji5i54aezv   portainer_agent     global       4/4        portainer/agent:lts
ii02x8cee6at   portainer_portainer  replicated   1/1        portainer/portainer-ce:lts   *:8000->8000/tcp, *:9000->9000/tcp, *:9443->9443/tcp
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose exec manager docker ps --filter name=portainer --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}'
NAMES                                          IMAGE                          STATUS
portainer_portainer.1.h75210yk9g7i719q6aj0335qn  portainer/portainer-ce:lts   Up 21 seconds
portainer_agent.vktrun1l2rvmov829r3pkq9fr.isijtnpif08a6yt1szfrqde8p  portainer/agent:lts  Up 23 seconds
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$
```

Figure 3.7. Un agent sur chaque machine (le manager et les 3 nœuds), et l'interface sur le manager.

3.2 Publier Portainer avec Traefik

J'ai ajouté le réseau `web` et les labels Traefik (port 9000) au service `portainer`, dans `portainer-agent-stack.traefik.yml`.

```
networks:
  - agent_network
  - web
deploy:
  # ...
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.portainer.rule=Host(`portainer.swarm.localhost`)"
    - "traefik.http.routers.portainer.entrypoints=web"
    - "traefik.http.services.portainer.loadbalancer.server.port=9000"
```

- `docker compose cp portainer-agent-stack.traefik.yml manager:/home/manager/`
- `docker compose exec manager docker stack deploy --with-registry-auth -c /home/manager/portainer-agent-stack.traefik.yml portainer`
- `docker compose exec manager docker stack services portainer`

3.3 Ouvrir Portainer dans le navigateur

```
127.0.0.1   portainer.swarm.localhost
```

Au premier lancement, Portainer demande le setup token qu'il écrit dans ses logs.

- `docker compose exec manager docker service logs portainer_portainer`

J'ai copié la valeur de `setup_token=` dans le formulaire, puis créé le compte `admin`.

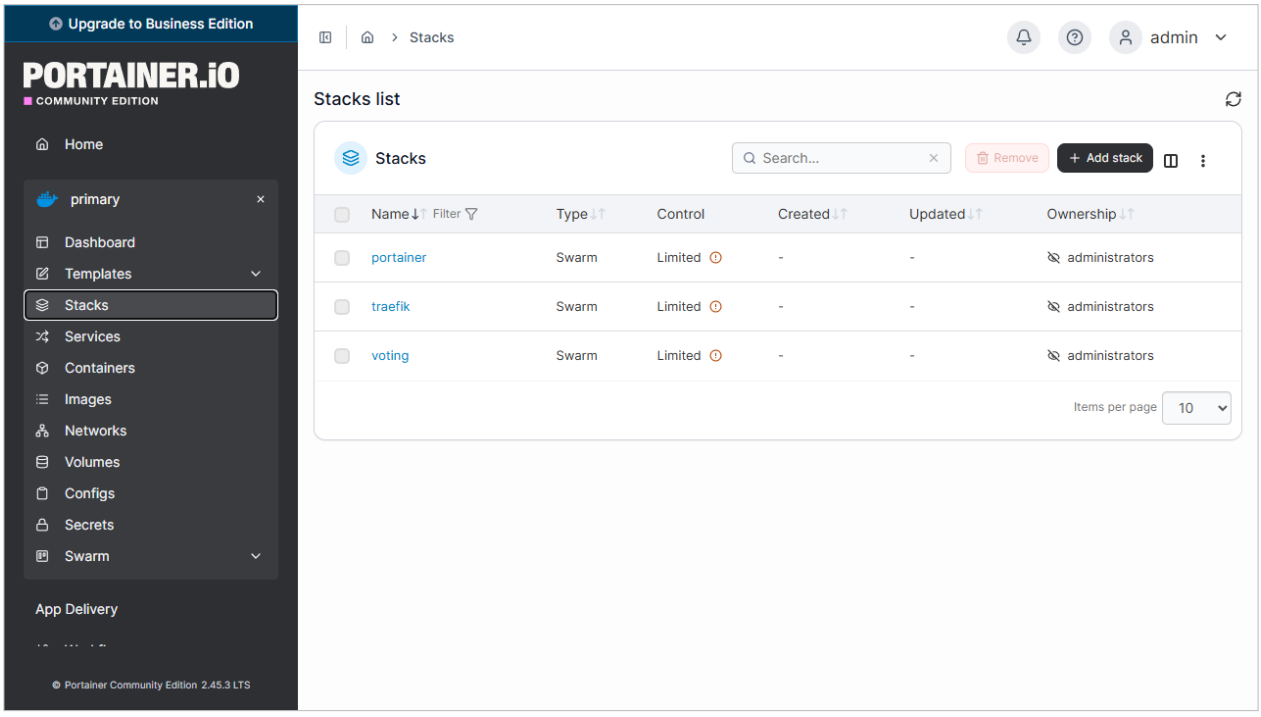


Figure 3.8. Je retrouve mes 3 stacks, en contrôle « Limited » car créées en ligne de commande.

Exercice 4 : Redéployer l'app de votes

4.1 Supprimer la stack depuis le manager

- `docker compose exec manager docker stack ls`
- `docker compose exec manager docker stack rm voting`
- `docker compose exec manager docker stack ls`

`voting` n'apparaît plus dans le second `docker stack ls`.

4.2 Republier la stack depuis Portainer

J'ai envoyé le fichier `docker-stack.traefik.yml` de la question 2.3.

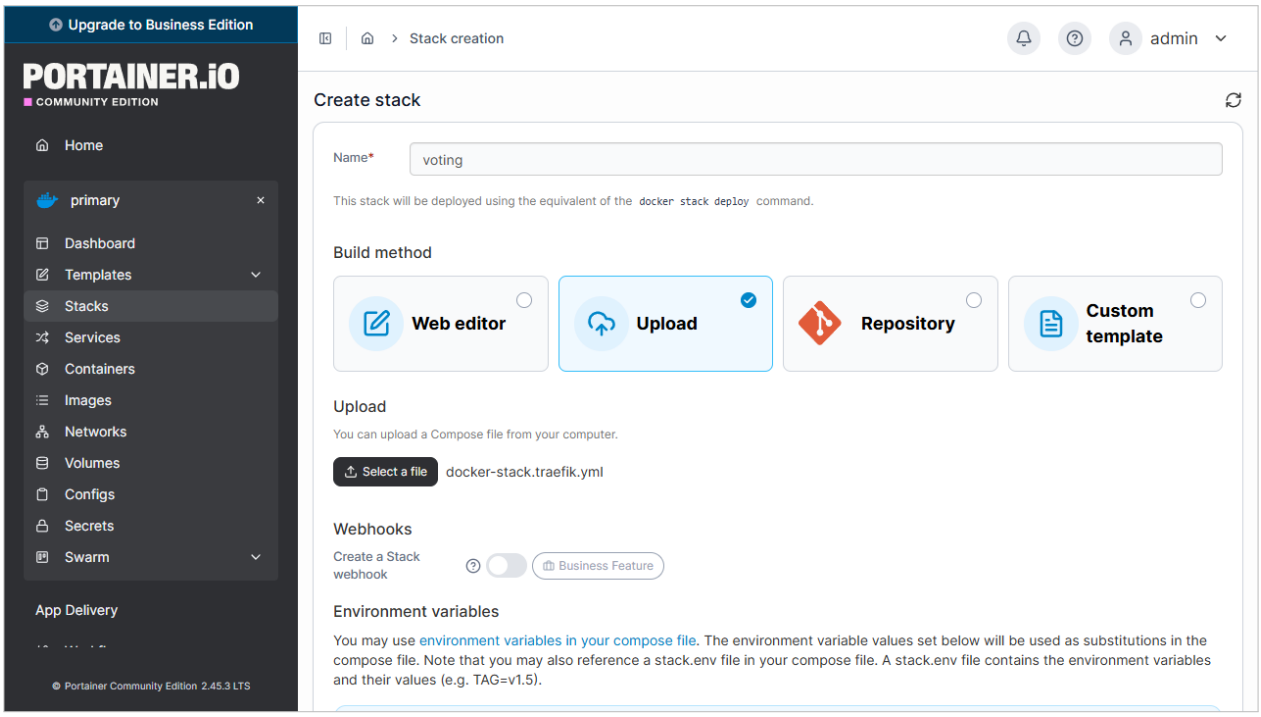


Figure 3.9. Création de la stack `voting` dans Portainer, par envoi du fichier.

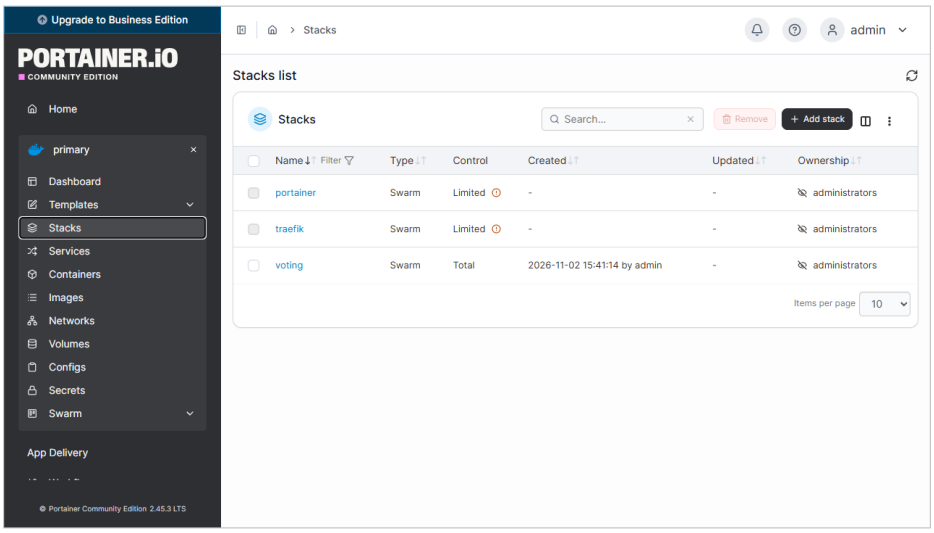


Figure 3.10. `voting` est maintenant en contrôle « Total ».

4.3 Vérifier l'accès depuis le navigateur

`vote.swarm.localhost` et `result.swarm.localhost` répondent de nouveau. Les résultats affichent « No votes yet » : la base a été recréée sur `node2` avec un volume neuf, les anciens votes sont dans le volume de `node3`.



Figure 3.11. `result.swarm.localhost` après le redéploiement depuis Portainer.

4.4 Déployer l'API de MFP

4.4.a Création de la stack

Le serveur utilise l'image publiée sur GHCR par mon workflow GitHub Actions, et passe par Traefik au lieu de publier le port 3000.

```yaml
server:
  # ...
  deploy:
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.mfp-api.rule=Host(`api.swarm.localhost`)"
      - "traefik.http.routers.mfp-api.entrypoints=web"
      - "traefik.http.services.mfp-api.loadbalancer.server.port=3000"
```

J'ai créé la stack `mfp` depuis Portainer, comme en 4.2.

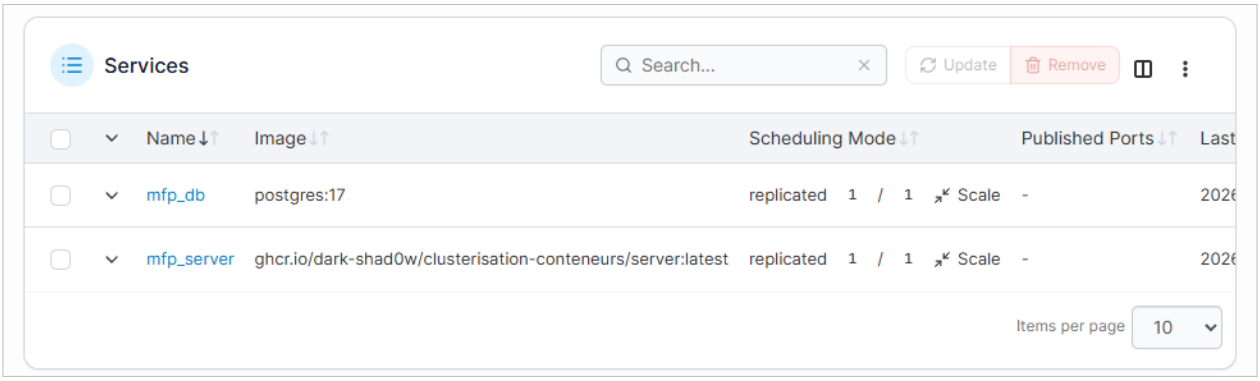


Figure 3.12. La base et le serveur de la stack `mfp` tournent, sans port publié.

Problème rencontré : au premier démarrage, le serveur s'est arrêté sur `getaddrinfo ENOTFOUND db`, la base n'étant pas encore prête. Swarm l'a relancé et il s'est connecté au deuxième essai.

4.4.b Test avec Bruno

```
127.0.0.1   api.swarm.localhost
```

J'ai fait une copie de la collection Bruno et changé son adresse avec `sed`.

- `cd ..`
- `cp -r mfp-bruno-api-test-collection ~/mfp-bruno-swarm`
- `sed -i 's#http://localhost:3000#http://api.swarm.localhost#' ~/mfp-bruno-swarm/*.yml`
- `cd ~/mfp-bruno-swarm && bru run`

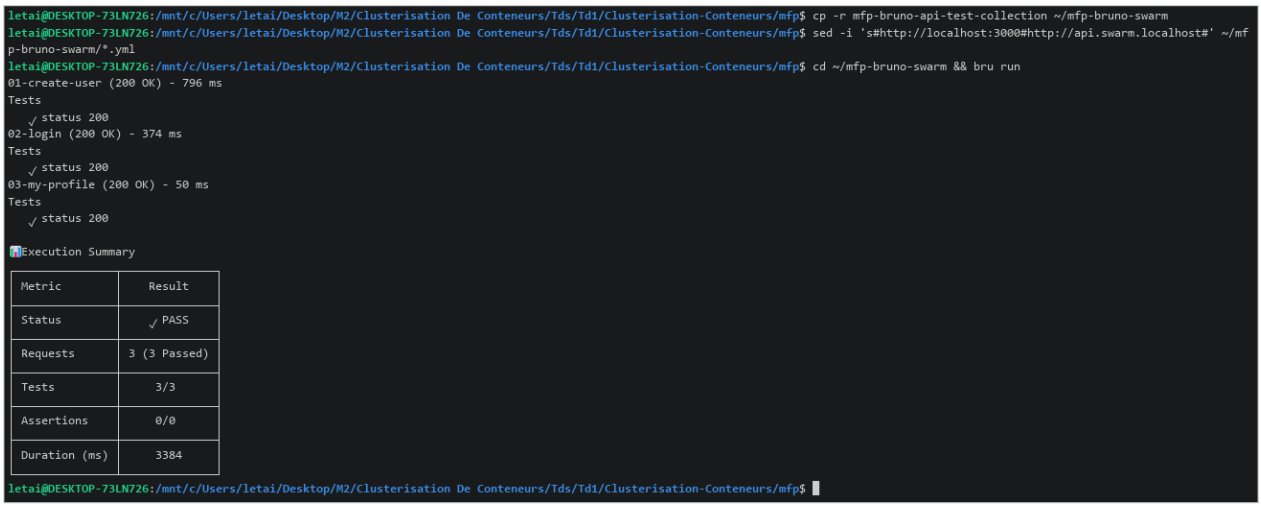


Figure 3.13. Les 3 requêtes de la collection passent sur `api.swarm.localhost`.

TD 4 : Secrets et configs Docker Swarm

Prérequis : relancer le cluster et passer en HTTPS

0.1 Relancer le cluster

Authelia n'accepte qu'une adresse en `https://`, j'ai donc ajouté le port 443 au manager.

```
manager:
  # ...
  ports:
    - "80:80"
    - "443:443"
```

- `cd mfp/swarm`
- `docker compose up -d`
- `docker compose ps`
- `docker compose exec manager docker node ls`

Problème rencontré : le manager a été recréé et `node1` à `node3` sont passés en `Down`, car les adresses IP avaient changé. J'ai fait quitter le cluster à toutes les machines et je l'ai recréé.

- `docker compose exec node1 docker swarm leave --force`
- `docker compose exec node2 docker swarm leave --force`
- `docker compose exec node3 docker swarm leave --force`
- `docker compose exec manager docker swarm leave --force`
- `docker compose exec manager docker swarm init`
- `TOKEN=$(docker compose exec manager docker swarm join-token -q worker)`
- `docker compose exec node1 docker swarm join --token $TOKEN manager:2377`
- `docker compose exec node2 docker swarm join --token $TOKEN manager:2377`
- `docker compose exec node3 docker swarm join --token $TOKEN manager:2377`
- `docker compose exec manager docker node ls`

Le manager et les 3 workers sont `Ready`. Le réseau `web` et les stacks ont disparu avec l'ancien cluster, mais les volumes sont restés sur les nœuds.

0.2 Créer le certificat

Sur mon PC, dans `mfp/swarm` :

- `mkdir certs`
- `cd certs`
- `openssl req -x509 -newkey rsa:2048 -noenc -subj "/CN=Swarm Local CA" -keyout ca.key -out ca.crt`
- `openssl req -newkey rsa:2048 -noenc -subj "/CN=*.swarm.localhost" -keyout swarm.localhost.key -out swarm.localhost.csr`

Le navigateur refuse un certificat dont le nom n'est pas dans `subjectAltName`. Je l'ai mis dans un fichier `certs/san.ext`.

```
subjectAltName=DNS:*.swarm.localhost
```

- `openssl x509 -req -in swarm.localhost.csr -CA ca.crt -CAkey ca.key -extfile san.ext -out swarm.localhost.crt`
- `openssl verify -CAfile ca.crt swarm.localhost.crt`
- `certutil.exe -user -addstore Root ca.crt`
- `cd ..`

`openssl verify` répond `swarm.localhost.crt: OK`, et `certutil` ajoute ma CA aux autorités de confiance de Windows.

0.3 Mettre le certificat dans un secret et des configs

Sur mon PC, j'ai copié les fichiers dans le manager, puis ouvert un shell dedans.

- `docker compose cp certs manager:/home/manager/`
- `docker compose cp traefik-tls.yml manager:/home/manager/`
- `docker compose exec manager ash`

Dans le manager, la clé privée va dans un secret, le certificat et `traefik-tls.yml` dans des configs. J'ai aussi recréé le réseau `web`.

- `cd /home/manager`
- `docker network create --driver overlay web`
- `docker secret create swarm_localhost_key certs/swarm.localhost.key`
- `docker config create swarm_localhost_crt certs/swarm.localhost.crt`
- `docker config create traefik_tls traefik-tls.yml`
- `docker config inspect --pretty traefik_tls`
- `docker secret inspect --pretty swarm_localhost_key`

```
/home/manager # docker config inspect --pretty traefik_tls
ID:                     l8ofn4slrx6jo1rwyd7gn2dau
Name:                   traefik_tls
Created at:             2026-11-04 09:22:27.882078733 +0000 utc
Updated at:             2026-11-04 09:22:27.882078733 +0000 utc
Data:
# Traefik dynamic configuration (file provider), stored in the swarm as a config.
# Default certificate for every https router: our local *.swarm.localhost certificate.
tls:
  stores:
    default:
      defaultCertificate:
        certFile: /certs/swarm.localhost.crt # config swarm_localhost_crt
        keyFile: /run/secrets/swarm.localhost.key # secret swarm_localhost_key
/home/manager # docker secret inspect --pretty swarm_localhost_key
ID:             gdhlcxzr9lw9z8je5bstsbucr
Name:             swarm_localhost_key
Driver:
Created at:       2026-11-04 09:22:28.648153145 +0000 utc
Updated at:       2026-11-04 09:22:28.648153145 +0000 utc
/home/manager #
```

Figure 4.1. La config affiche son contenu, le secret seulement son nom et ses dates.

0.4 Passer Traefik en HTTPS

Dans `traefik-https.stack.yml`, j'ai ajouté l'entrypoint `websecure` sur le port 443, les configs, le secret, et le provider `file` qui lit `traefik-tls.yml`. L'alias `authelia.swarm.localhost` sert à la question 1.2.

```yaml
networks:
  web:
    aliases:
      - authelia.swarm.localhost
ports:
  - 80:80
  - 443:443
configs:
  - source: traefik_tls
    target: /etc/traefik/tls.yml
  - source: swarm_localhost_crt
    target: /certs/swarm.localhost.crt
secrets:
  - source: swarm_localhost_key
    target: swarm.localhost.key
command:
  # ...
  - "--entrypoints.websecure.address=:443"
  - "--entrypoints.websecure.http.tls=true"
  - "--providers.file.filename=/etc/traefik/tls.yml"
```

Dans la même stack, `whoami` écoute aussi sur `websecure`, et un routeur `to-https` de priorité 1 renvoie vers `https://` les adresses qui n'ont pas de routeur en HTTP.

Sur mon PC :

- `docker compose cp traefik-https.stack.yml manager:/home/manager/`

Dans le manager :

- `docker stack deploy -c traefik-https.stack.yml traefik`
- `docker service ls`

Sur mon PC :

- `curl -I --cacert certs/ca.crt https://whoami.swarm.localhost`
- `curl -I http://dozzle.swarm.localhost`

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conten
eurs/mfp/swarm$ curl -I --cacert certs/ca.crt https://whoami.swarm.localhost
HTTP/2 200
content-type: text/plain; charset=utf-8
date: Thu, 05 Nov 2026 11:40:17 GMT
content-length: 359
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conten
eurs/mfp/swarm$ curl -I http://dozzle.swarm.localhost
HTTP/1.1 307 Temporary Redirect
Location: https://dozzle.swarm.localhost/
Date: Thu, 05 Nov 2026 11:40:37 GMT
Content-Length: 18
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conten
eurs/mfp/swarm$
```

Figure 4.2. `whoami` répond en HTTPS (`HTTP/2 200`) avec mon certificat, et `http://dozzle.swarm.localhost`, sans routeur HTTP, est redirigé (307) vers `https://`.

0.5 Relancer Portainer et ajouter les adresses

La stack de Portainer était encore dans `/home/manager` depuis le TD 3. Dans le manager :

- `docker stack deploy -c portainer-agent-stack.traefik.yml portainer`

Portainer a retrouvé son compte `admin`, resté dans son volume sur le manager. J'ai ajouté les trois adresses du TD au fichier hosts.

```
127.0.0.1   dozzle.swarm.localhost
127.0.0.1   authelia.swarm.localhost
127.0.0.1   outline.swarm.localhost
```

Exercice 1 : Déployer des logiciels

1.1 Dozzle avec un utilisateur simple

1.1.a Générer users.yml

Sur mon PC, avec la commande de la documentation de Dozzle :

- `docker run --rm amir20/dozzle generate admin --password dozzle-admin-2026 --name "Admin"`

Elle affiche le contenu de `users.yml`, que j'ai copié.

1.1.b Créer le secret dans Portainer

Dans Portainer, « Secrets » puis « Add secret » : nom `dozzle_users`, et le contenu copié juste avant.

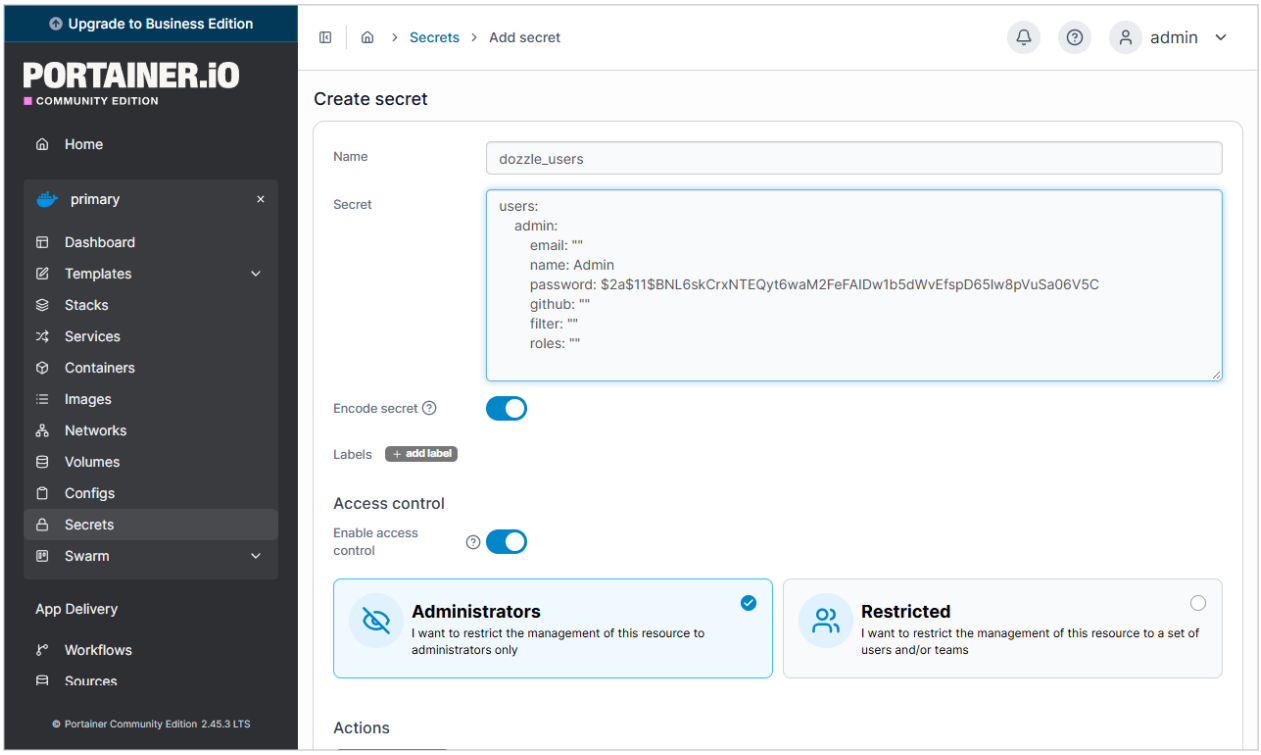


Figure 4.3. Création du secret `dozzle_users` dans Portainer.

1.1.c Déployer Dozzle

```
environment:
  DOZZLE_MODE: swarm
  DOZZLE_AUTH_PROVIDER: simple
# ...
secrets:
  - source: dozzle_users
    target: /data/users.yml
```

Sur mon PC :

- `docker compose cp dozzle.stack.yml manager:/home/manager/`

Dans le manager :

- `docker stack deploy -c dozzle.stack.yml dozzle`
- `docker stack services dozzle`

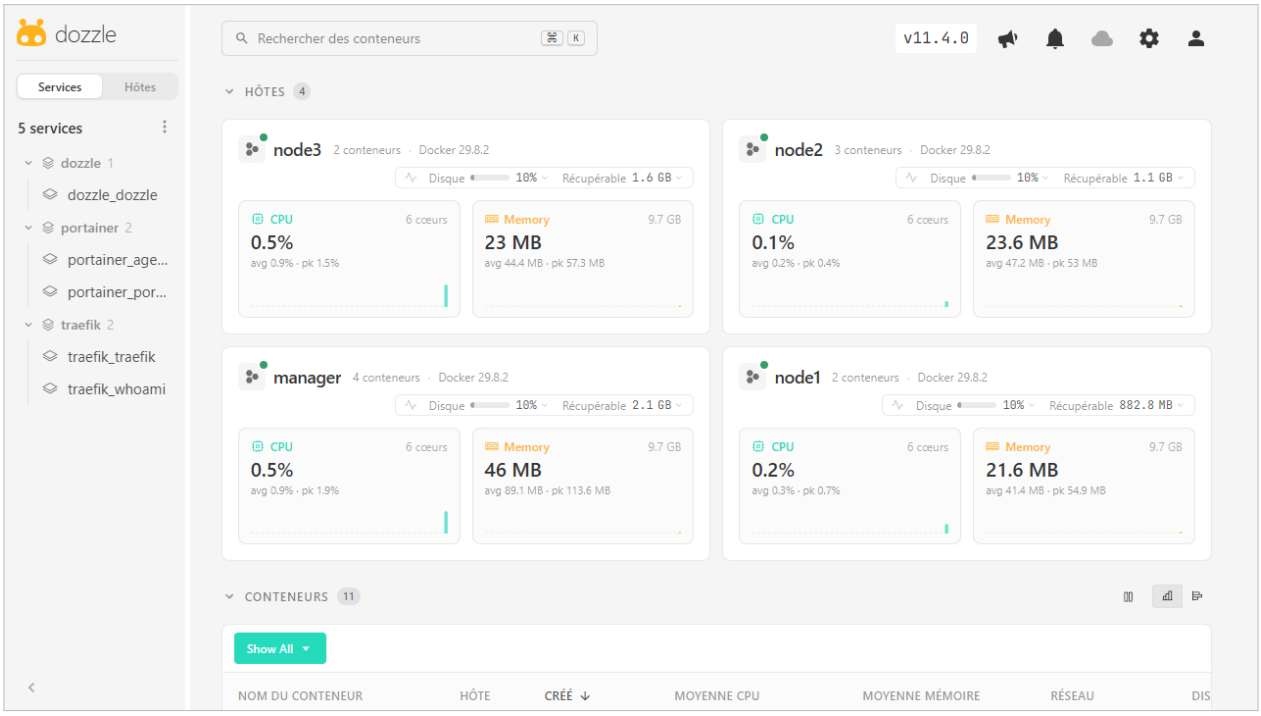


Figure 4.4. Connecté à `dozzle.swarm.localhost` avec le compte `admin`, Dozzle voit le manager et les 3 nœuds.

1.2 Authelia et connexion OIDC à Dozzle

1.2.a Créer les clés d'Authelia

Dans le manager, chaque clé sort d'`openssl` et part directement dans un secret. La config `swarm_ca` (ma CA) servira à Dozzle et Outline.

- `openssl rand -hex 32 | docker secret create authelia_jwt_secret -`
- `openssl rand -hex 32 | docker secret create authelia_session_secret -`
- `openssl rand -hex 32 | docker secret create authelia_storage_key -`
- `openssl rand -hex 32 | docker secret create authelia_oidc_hmac -`
- `openssl genrsa 2048 | docker secret create authelia_oidc_key -`
- `docker config create swarm_ca certs/ca.crt`

1.2.b Créer le compte dans un secret

Sur mon PC :

- `docker run --rm -it authelia/authelia:4.39 authelia crypto hash generate argon2`

La commande demande le mot de passe et affiche son hash (ligne `Digest`). Je l'ai collé dans `password` du fichier `secrets/users_database.yml`.

- `cat secrets/users_database.yml`
- `docker compose cp secrets/users_database.yml manager:/home/manager/`

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conten
eurs/mfp/swarm$ cat secrets/users_database.yml
users:
  ahmed:
    displayname: 'Ahmed'
    password: '$argon2id$v=19$m=65536,t=3,p=4$wIksLjZLI6+vY01GqMoWsQ$ahFnNv1pcu5dAPHvB8nwz2uGvGPomDFPdsuiGRwTZ
hM'
    email: 'ahmed@swarm.localhost'
    groups:
      - 'admins'
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conten
eurs/mfp/swarm$
```

Figure 4.5. Le compte `ahmed`, du groupe `admins`, avec son hash argon2.

Dans le manager :

- `docker secret create authelia_users users_database.yml`

1.2.c Créer le client OIDC de Dozzle

Sur mon PC :

- `docker run --rm authelia/authelia:4.39 authelia crypto hash generate pbkdf2 --random`

Elle affiche un mot de passe aléatoire (`Random Password`) et son empreinte (`Digest`). J'ai mis le mot de passe dans un secret `dozzle_oidc_secret`, créé dans Portainer comme `dozzle_users`, et l'empreinte dans `client_secret` du client `dozzle` d'`authelia.configuration.yml`.

1.2.d Déployer Authelia

```yaml
session:
  cookies:
    - domain: 'swarm.localhost'
      authelia_url: 'https://authelia.swarm.localhost'
# ...
    clients:
      - client_id: 'dozzle'
        client_secret: '$pbkdf2-sha512$310000$...'
        # ...
        redirect_uris:
          - 'https://dozzle.swarm.localhost/api/auth/callback'
```

Dans `authelia.stack.yml`, les clés sont lues avec les variables `AUTHELIA_*_FILE`, sauf la clé OIDC, que la configuration lit avec le filtre `template`.

```yaml
environment:
  X_AUTHELIA_CONFIG: /config/configuration.yml
  X_AUTHELIA_CONFIG_FILTERS: template
  # ...
  AUTHELIA_SESSION_SECRET_FILE: /run/secrets/authelia_session_secret
  AUTHELIA_STORAGE_ENCRYPTION_KEY_FILE: /run/secrets/authelia_storage_key
  # ...
configs:
  - source: authelia_config
    target: /config/configuration.yml
secrets:
  - source: authelia_users
    target: /config/users_database.yml
  - authelia_jwt_secret
  # ...
```

Sur mon PC :

- `docker compose cp authelia.configuration.yml manager:/home/manager/`
- `docker compose cp authelia.stack.yml manager:/home/manager/`

Dans le manager :

- `docker config create authelia_config authelia.configuration.yml`
- `docker stack deploy -c authelia.stack.yml authelia`
- `docker stack services authelia`

```
/home/manager # docker stack services authelia
ID             NAME                MODE         REPLICAS   IMAGE                   PORTS
nbui8gga4l2a   authelia_authelia   replicated   1/1        authelia/authelia:4.39
/home/manager #
```

Figure 4.6. `authelia_authelia` est en `1/1`.

1.2.e Brancher Dozzle sur Authelia

Dozzle joint `authelia.swarm.localhost` par l'alias de Traefik, et la config `swarm_ca` lui fait accepter mon certificat.

```yaml
environment:
  DOZZLE_AUTH_PROVIDER: oidc
  DOZZLE_AUTH_OIDC_ISSUER: https://authelia.swarm.localhost
  DOZZLE_AUTH_OIDC_CLIENT_ID: dozzle
  DOZZLE_AUTH_OIDC_CLIENT_SECRET_FILE: /run/secrets/dozzle_oidc_secret
  SSL_CERT_FILE: /certs/ca.crt
# ...
configs:
  - source: swarm_ca
    target: /certs/ca.crt
```

Sur mon PC :

- `docker compose cp dozzle.oidc.stack.yml manager:/home/manager/`

Dans le manager :

- `docker stack deploy -c dozzle.oidc.stack.yml dozzle`

Problème rencontré : au retour d'Authelia, Dozzle refusait la connexion avec `no roles claim found in the ID token`. J'ai ajouté à Authelia un attribut `dozzle_roles`, calculé à partir des groupes, et Dozzle le demande avec `DOZZLE_AUTH_OIDC_SCOPES: dozzle_roles`.

```
definitions:
  user_attributes:
    dozzle_roles:
      expression: '"admins" in groups ? ["all"] : []'
# ...
    claims_policies:
      dozzle:
        id_token: ['dozzle_roles']
        # ...
```

Une config ne se modifie pas, il faut la supprimer puis la recréer. Sur mon PC :

- `docker compose cp authelia.configuration.yml manager:/home/manager/`
- `docker compose cp dozzle.oidc.stack.yml manager:/home/manager/`

Dans le manager :

- `docker config rm authelia_config`

Problème rencontré : la suppression a été refusée, parce que la config était encore utilisée par Authelia. J'ai d'abord retiré la stack `authelia`, sans toucher à son volume.

- `docker stack rm authelia`
- `docker config rm authelia_config`
- `docker config create authelia_config authelia.configuration.yml`
- `docker stack deploy -c authelia.stack.yml authelia`
- `docker stack deploy -c dozzle.oidc.stack.yml dozzle`

1.2.f Se connecter à Dozzle par Authelia

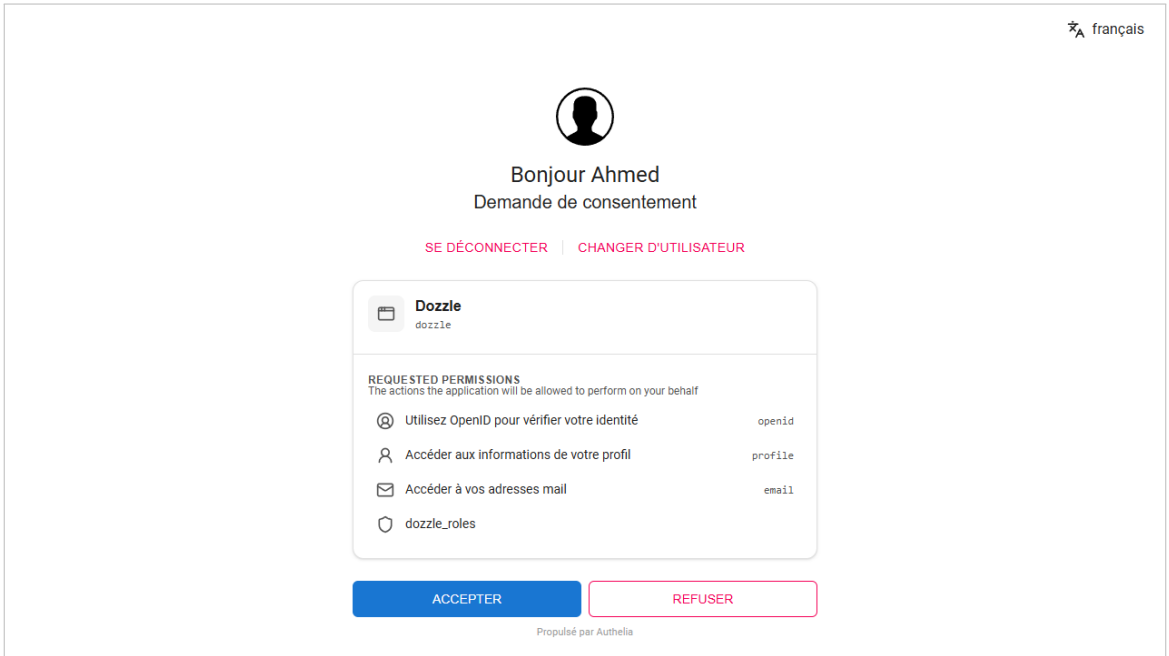


Figure 4.7. Connecté à Authelia avec le compte `ahmed`, qui demande mon accord pour Dozzle.

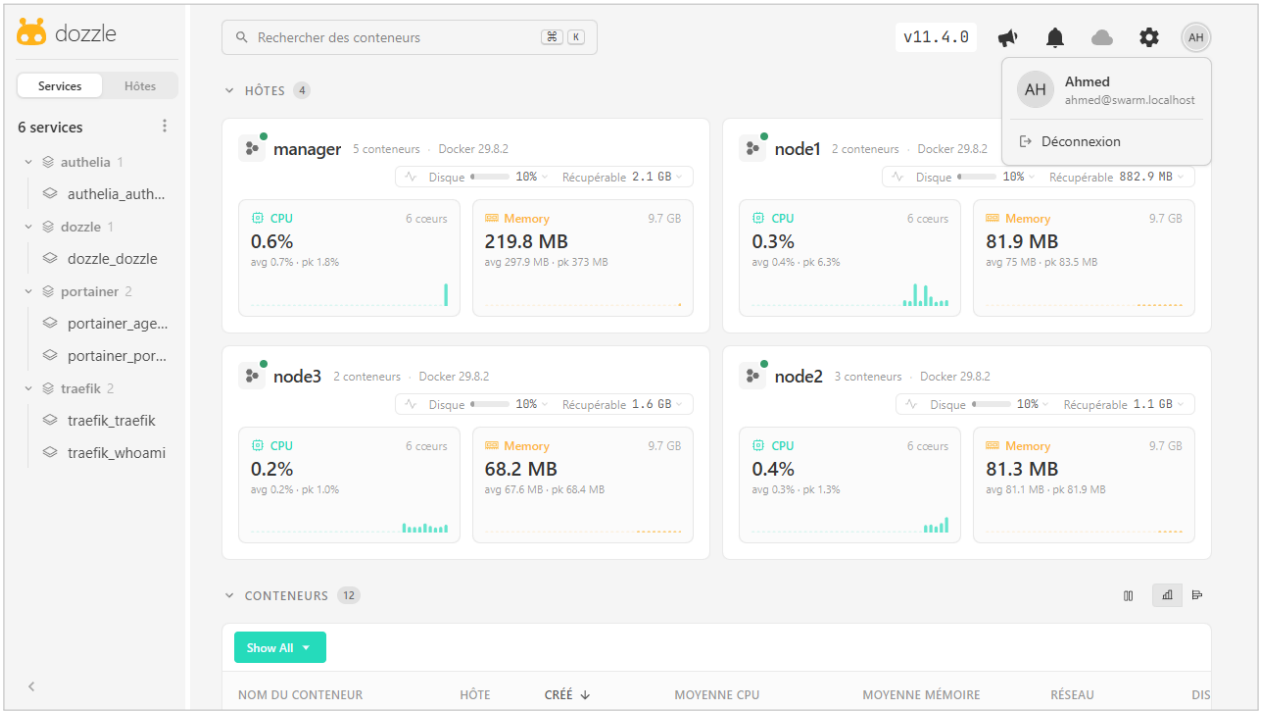


Figure 4.8. De retour sur Dozzle, connecté en tant qu'Ahmed, le compte du secret `authelia_users`.

1.3 Bonus : Outline

1.3.a Créer les secrets d'Outline

Dans le manager :

- `openssl rand -hex 32 | docker secret create outline_secret_key -`
- `openssl rand -hex 32 | docker secret create outline_utils_secret -`
- `openssl rand -hex 24`

Le dernier `openssl` affiche le mot de passe de PostgreSQL. Je l'ai collé dans deux secrets créés dans Portainer : `outline_db_password`, avec le mot de passe seul, et `outline_database_url`, avec `postgres://outline:<mot de passe>@postgres:5432/outline`.

Pour le client OIDC, sur mon PC :

- `docker run --rm authelia/authelia:4.39 authelia crypto hash generate pbkdf2 --random`

Comme pour Dozzle, le mot de passe va dans un secret `outline_oidc_secret` créé dans Portainer, et l'empreinte dans `client_secret` du client `outline`.

1.3.b Ajouter le client dans Authelia

```
- client_id: 'outline'
  client_name: 'Outline'
  # ...
  redirect_uris:
    - 'https://outline.swarm.localhost/auth/oidc.callback'
```

Comme demandé, j'ai supprimé puis recréé la config, en retirant la stack avant cette fois. Sur mon PC :

- `docker compose cp authelia.configuration.yml manager:/home/manager/`

Dans le manager :

- `docker stack rm authelia`
- `docker config rm authelia_config`
- `docker config create authelia_config authelia.configuration.yml`
- `docker stack deploy -c authelia.stack.yml authelia`

Cette fois, `config rm` est passé du premier coup.

1.3.c Déployer Outline

```yaml
environment:
  # ...
  SECRET_KEY_FILE: /run/secrets/outline_secret_key
  UTILS_SECRET_FILE: /run/secrets/outline_utils_secret
  DATABASE_URL_FILE: /run/secrets/outline_database_url
  # ...
  OIDC_CLIENT_ID: outline
  OIDC_CLIENT_SECRET_FILE: /run/secrets/outline_oidc_secret
  OIDC_AUTH_URI: https://authelia.swarm.localhost/api/oidc/authorization
  OIDC_TOKEN_URI: https://authelia.swarm.localhost/api/oidc/token
  OIDC_USERINFO_URI: https://authelia.swarm.localhost/api/oidc/userinfo
  # ...
  NODE_EXTRA_CA_CERTS: /certs/ca.crt
```

Sur mon PC :

- `docker compose cp outline.stack.yml manager:/home/manager/`

Dans le manager :

- `docker stack deploy -c outline.stack.yml outline`
- `docker stack services outline`

```
/home/manager # docker stack services outline
ID             NAME               MODE         REPLICAS   IMAGE                          PORTS
n1c3g1io02je   outline_outline    replicated   1/1        outlinewiki/outline:1.12.0
qlu54b1aog9g   outline_postgres   replicated   1/1        postgres:17
usq34npjpqqi   outline_redis      replicated   1/1        redis:7-alpine
/home/manager #
```

Figure 4.9. `outline`, `postgres` et `redis` sont en `1/1`.

1.3.d Se connecter à Outline par Authelia

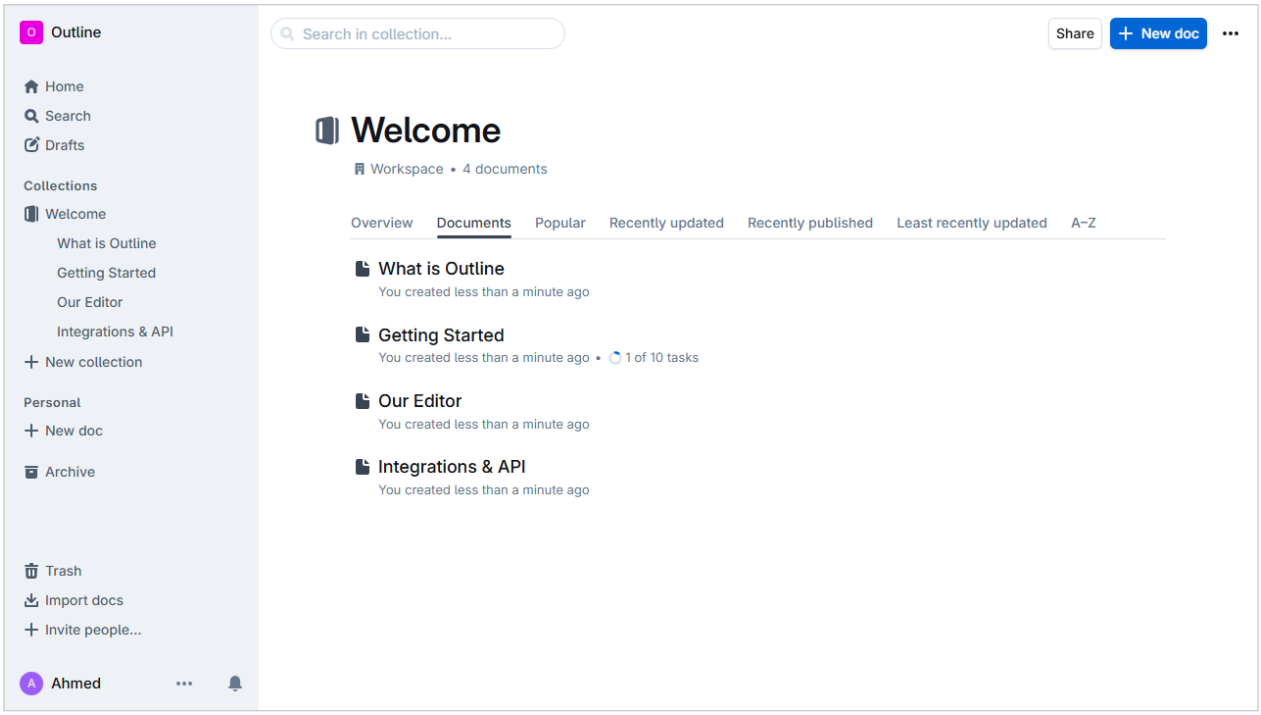


Figure 4.10. Après la connexion sur Authelia, Outline s'ouvre sur `outline.swarm.localhost`, connecté en tant qu'Ahmed.